

1 Design

Syscalls are generally implemented in the most general way possible in order to allow for maximum flexibility in their use. This is a little more difficult to design because for the purposes of this project, we were to implement the syscall as a by-product of the userspace `ls` program.

I considered several syscall designs: a syscall that designs the output for the `ls` program, call it from userspace and print the output. On the other extreme, a syscall that returns as much data as is reasonable. The reasonable middleground is to write a versatile syscall and then select the required data and format it in userspace. This means that more memory is used than strictly necessary, but that the syscall is more versatile and can be used for other programs (in this case, such as `tree`, `find` etc).

The next consideration was which article to use as the unique identifier of each file. Regardless of type, this could either be the path (which is unique) or the file object. For this implementation, I used the file object. The overarching reason for this is that the internal representation of files/directories in `stacos` is the file object. This means that my implementation leaves complete flexibility of potential changes in the vfs layer, as long as the lookup method and 'open' syscall implementation is changed accordingly. However, it follows from these assumptions that using the path and the lookup method also amounts to the same thing in the current implementation. The exception is if pathing changes (such as if symlinks are implemented). In such a case, the file object : file relationship would be a stronger bound than the path : file relationship.

Next, the userspace program needed to be designed. At the core it is quite simple – it simply parses the command line to get the flags and the path, then uses two booleans in order to store the use of the flags. It then performs memcpy operations to copy instances of `stat1` out of the buffer given to the syscall. Based on the flags, it lazily formats and prints the output to the console.

2 Syscall Implementation

Keeping in mind the design considerations detailed above, this implementation tries to keep user-space code to a minimum and limit information being processed or augmented. The syscall is implemented to take in a buffer, an offset, and the length of the buffer. If the buffer is not large enough to hold all data, the syscall will return data based on what is useful rather than just what the userspace program needs. There are three key elements of this implementation:

- The system call: the syscall is not made from the `ls` program, but from the userspace 'object' class. It returns an exit code, but the actual information is communicated by the kernel through the syscall using the buffer. The exit code is used to prevent errors – for example, it can tell the userspace program if the path given to the program leads to a file, which helps prevent segmentation faults.
- The 'stat1' class: 'stat1' (stat-like from the linux 'stat' struct) is a class containing file metadata without the boilerplate of 'fs_node' and its subclasses. The implementation may be added to by a potential `stacos` developer, who needs more file metadata or functionality. It is also usable in other cases where metadata is needed
- Functions associated with the syscall: The implementation of the syscall itself exists in the `syscall.cpp` file. The rest of the operation of the syscall is implemented largely in a function that is called on instances of the `fat_file` class. Its implementation in `fat.cpp` loops over `children_` in the `fat_node` associated with the `fat_file` instance in question. It constructs 'stat1' for each child, then copies it into the memory buffer. Each 'directory entry' is fixed-size, so we can retrieve it easily. After looping over all directories, the function null-terminates the buffer. The buffer must be sized appropriately. If it is not, the function returns 1 (as error code).

3 Userspace Program

The userspace program uses the 'dirents' syscall (which is generalised) and filters out the data that it needs to display. This is done by using the memcpy operation, with the destination being an empty instance of the 'stat1' class. Based on the CLI flags, it will extract the required information. It then formats that information in the correct way and prints it. The `ls` command calls the `dirents` method of object on an opened file object, which writes a `stat1` entry to the buffer for each of the files in the directory (note that these files may be directories, or 'regular files' in the layperson sense). The formatting is dependent on the flags with which the command is called. This implementation allows two flags – '-l' (long output) and '-a' (show hidden files).

4 Testing

As with other `stacos` programs, the `ls` command is generally use-case-testing. To do this, I ran the '/usr/ls' command with all possible flag combinations (-a -l, -al, -la, -l...) on a few different files. The files I used were /tree, /usr, /, /tree/c and /usr/cat. Of these, the expected behaviour on /usr/cat would be for the program to print out an error message, then exit safely without erring or segfaulting. For all other files – `ls` will print all subdirectories and files. The -l flag prints out all non-hidden files/directories and for all regular files, prints size of the file contents. The -a flag prints out all hidden files, and a combination of the two should do both extensions. Hidden files and directories are simply those whose names begin with '.' (such as '.hidden' in /tree/c/)

This implementation behaves as expected in all scenarios described above.